

De la interpolación clásica a la Super-Resolución con Deep Learning y Transformers

Documentación técnica complementaria al notebook [Zoom_Resize_Image.ipynb](#).

El notebook cubre los fundamentos (píxel, PPI/DPI) y la interpolación clásica (Nearest, Bilineal, Bicúbica, Lanczos). Este documento profundiza en **por qué** esos métodos tienen un techo de calidad y en las técnicas modernas de **Super-Resolución de Imagen Única** (Single Image Super-Resolution, **SISR**): desde la idea de "aprender de ejemplos" (RAISR), pasando por las redes convolucionales y los GAN (Real-ESRGAN), hasta los **Transformers** (SwinIR / Swin2SR).

Índice

1. Recordatorio: el problema del reescalado
 2. Interpolación clásica: la matemática y su techo
 3. Formulación de la Super-Resolución
 4. El modelo de degradación (degradation pipeline)
 5. Super-Resolución basada en ejemplos: del diccionario a RAISR
 6. Super-Resolución con redes convolucionales (CNN)
 7. El salto perceptual: SRGAN, ESRGAN y Real-ESRGAN
 8. Transformers para imágenes: ViT, SwinIR y Swin2SR
 9. Métricas de calidad y el dilema percepción–distorsión
 10. Notas prácticas (Colab / GPU T4)
 11. Referencias
-

1. Recordatorio: el problema del reescalado

Ampliar (*upscaling*) una imagen significa estimar los valores de píxeles que **no existían** en la imagen original. Si una imagen de baja resolución (LR) de tamaño $W \times H$ se amplía un factor s , hay que generar $s^2 \cdot W \cdot H - W \cdot H$ píxeles nuevos. El reescalado es, por tanto, un **problema mal planteado** (*ill-posed*): para una misma imagen LR existen infinitas imágenes de alta resolución (HR) que, al reducirse, producirían esa LR. Cualquier método de ampliación no es más que una forma de **elegir una** de esas infinitas soluciones.

- La **interpolación clásica** elige asumiendo que la imagen es *suave* (los píxeles nuevos son combinaciones de los vecinos). Es una *prior* matemática fija.
 - La **super-resolución aprendida** elige usando una *prior* aprendida de datos reales: "las imágenes naturales tienen bordes, texturas, patrones de piel/madera/pelo...". Por eso puede "alucinar" detalle plausible que la interpolación nunca generaría.
-

2. Interpolación clásica: la matemática y su techo

El notebook compara cuatro métodos. Aquí va la base formal de cada uno.

Nearest-Neighbor

Asigna a cada píxel destino el valor del píxel origen más cercano. Equivale a convolucionar con un núcleo rectangular (*box*). Es **0(1)** por píxel; preserva bordes duros pero produce el clásico efecto de "bloques" (*aliasing*).

Bilineal

Media ponderada de los **4** vecinos más próximos (interpolación lineal en X y en Y). Núcleo triangular (*tent*). Suaviza, pero atenúa altas frecuencias → emborrona.

Bicúbica

Media ponderada de un vecindario **4×4** (16 píxeles) usando un polinomio cúbico por tramos (núcleo de Keys, normalmente con **a = -0.5**). Mejor preservación de detalle que bilineal; es el estándar de facto para fotografía.

Lanczos

Usa una **sinc enventanada** (núcleo $\text{sinc}(x) \cdot \text{sinc}(x/a)$ con **a = 2** o **3**). El **sinc** es el reconstructor ideal para una señal con ancho de banda limitado (teorema de Nyquist–Shannon), por lo que Lanczos da los bordes más nítidos de los cuatro, a costa de posibles halos (*ringing*) y mayor coste.

El techo común

Todos son **filtros lineales de convolución con un núcleo fijo**. La salida es una combinación lineal de píxeles de entrada:

$$I_{HR}(x, y) = \sum_{i,j} k(i, j) I_{LR}(x_i, y_j)$$

Como **k** no depende del contenido de la imagen, **no puede añadir frecuencias nuevas**: solo redistribuye la información existente. Matemáticamente, la energía en altas frecuencias que se perdió al reducir la imagen **no se puede recuperar** con un filtro lineal. De ahí el emborronamiento inevitable. Romper ese techo exige una *prior* no lineal y dependiente del contenido: eso es lo que aporta el aprendizaje.

📖 Lectura recomendada sobre Lanczos: <https://mazzo.li/posts/lanczos.html>

3. Formulación de la Super-Resolución

La SISR busca una función **F** que, dada una imagen LR, estime la HR:

$$\hat{I}_{HR} = F(I_{LR}; \theta)$$

donde **θ** son parámetros aprendidos minimizando una pérdida sobre un conjunto de pares (**I_{LR}**, **I_{HR}**):

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(F(I_{LR}^{(n)}; \theta), I_{HR}^{(n)})$$

La pieza clave es: **¿de dónde salen esos pares de entrenamiento?** Casi nunca se tienen fotos reales del mismo objeto a dos resoluciones perfectamente alineadas. La solución —exactamente la intuición de

"downgradear imágenes buenas"— es el modelo de degradación.

4. El modelo de degradación (degradation pipeline)

Se parte de imágenes HR de alta calidad y se generan sus versiones LR aplicando una degradación conocida. El modelo clásico ("bicubic degradation") es:

$$I_{LR} = (I_{HR} * k) \downarrow_s$$

es decir, **desenfoque** (convolución con un núcleo k) seguido de **submuestreo** por un factor s . El modelo realista (BSRGAN / Real-ESRGAN) lo amplía con una cadena más rica:

$$I_{LR} = [(I_{HR} * k) \downarrow_s + n]_{\text{JPEG}}$$

añadiendo **ruido** n (gaussiano, de Poisson, del sensor) y **artefactos de compresión JPEG**, a menudo aplicados varias veces ("*high-order degradation*") para imitar imágenes del mundo real (capturas, fotos antiguas, vídeo recomprimido).

En el notebook, la función `degrade()` implementa la versión simple (achicar + reampliar con bicúbica) para ilustrar el concepto y generar el par de entrenamiento de la "tabla".

Por qué importa: el modelo aprendido solo sabe deshacer las degradaciones que vio en entrenamiento. Un modelo entrenado solo con "bicubic degradation" funciona mal con fotos reales ruidosas; por eso Real-ESRGAN invierte tanto esfuerzo en una degradación realista.

5. Super-Resolución basada en ejemplos: del diccionario a RAISR

Es la familia a la que pertenece, conceptualmente, la "tabla de vecindarios" del notebook.

Example-based SR (Freeman et al., 2002)

Se construye una base de datos de **parches** (*patches*) emparejados LR↔HR. Para ampliar, por cada parche LR de la imagen de entrada se busca el parche LR más parecido del diccionario y se pega su contraparte HR. Funciona, pero:

- La base de datos es enorme y la búsqueda del vecino más cercano es lenta.
- **La maldición de la dimensionalidad:** el número de configuraciones posibles de un vecindario crece exponencialmente con su tamaño. Con 8 vecinos en escala de grises ya hay $256^8 \approx 1.8 \cdot 10^{19}$ combinaciones; en color es astronómico. La tabla nunca se puede llenar ni almacenar → se queda vacía justo donde la consultas.

Sparse coding / dictionary learning (Yang et al., 2010)

En vez de guardar todos los parches, aprende un **diccionario compacto** y representa cada parche como combinación *sparse* de sus átomos. Comprime el problema, pero sigue siendo lento.

RAISR (Google, 2016) — la idea "apañada"

Rapid and Accurate Image Super-Resolution. En lugar de una tabla con todos los colores, clasifica cada parche en un número pequeño de **cubos (buckets)** según características de su **gradiente local**: ángulo,

fuerza y coherencia. Para cada cubo aprende un **filtro lineal óptimo** (por mínimos cuadrados) a partir de los pares LR/HR. En inferencia: calcula el gradiente, elige el cubo, aplica su filtro. Resultado: calidad cercana a las CNN de su época, pero **órdenes de magnitud más rápido** y ligero.

RAISR es prácticamente la versión funcional de la "tabla de vecindarios con cajas". La mini-implementación del notebook (cuantizar el vecindario 3×3 en **Q** niveles y aprender el color medio por caja) es un RAISR de juguete: muestra que la idea funciona y por qué la **cuantización** (agrupar en pocas cajas) es lo que la hace viable.

👤 RAISR: <https://arxiv.org/abs/1606.01299>

6. Super-Resolución con redes convolucionales (CNN)

La idea de "aprender la *prior* de las imágenes naturales" cristaliza con el deep learning. Una CNN **no guarda una tabla**: guarda una función comprimida en sus pesos que **generaliza** a vecindarios nunca vistos. Hitos:

Modelo	Año	Aportación principal
SRCNN	2014	Primera CNN para SR. 3 capas: extracción de parches, mapeo no lineal, reconstrucción. Demostró que una red supera a la interpolación.
FSRCNN	2016	Procesa en resolución LR y amplía al final → mucho más rápido.
ESPCN	2016	Introduce el sub-pixel convolution / pixel shuffle : la red aprende s² mapas y los reordena para ampliar, sin interpolar antes. Estándar hoy.
VDSR	2016	Red muy profunda (20 capas) + aprendizaje residual (predice solo el detalle que falta).
EDSR	2017	Bloques residuales sin BatchNorm; ganador del reto NTIRE. Gran calidad en PSNR.
RCAN	2018	Atención de canal + redes muy profundas (400+ capas).

Funciones de pérdida. Las primeras CNN minimizan el error píxel a píxel:

$$\mathcal{L}_1 = \|I_{HR} - \hat{I}_{HR}\|_1 \quad \mathcal{L}_2 = \|I_{HR} - \hat{I}_{HR}\|_2^2$$

Maximizan PSNR pero tienden a dar imágenes **suaves** (la media de todas las HR plausibles es borrosa). Resolver eso lleva al siguiente salto.

7. El salto perceptual: SRGAN, ESRGAN y Real-ESRGAN

SRGAN (2017)

Introduce dos ideas para imágenes **perceptualmente** nítidas en vez de óptimas en PSNR:

- **Pérdida perceptual** (*perceptual / content loss*): compara *features* de una red preentrenada (VGG) en lugar de píxeles, premiando que las texturas "parezcan" correctas.
- **Pérdida adversaria** (GAN): un **discriminador** **D** aprende a distinguir HR reales de generadas, y el generador **G** aprende a engañarlo. El discriminador es, en esencia, el "juez que detecta inventos que no parecen reales" → es el equivalente aprendido del sistema de "filtros vigilantes por consenso".

$$\mathcal{L}_G = \underbrace{\mathcal{L}_{perceptual}}_{\text{textura realista}} + \lambda \underbrace{\mathcal{L}_{adv}}_{\text{engañar a } D}$$

ESRGAN (2018)

Mejora SRGAN con bloques **RRDB** (Residual-in-Residual Dense Block) sin BatchNorm, un **discriminador relativista** (juzga *cuánto más realista* es una imagen que otra) y pérdida perceptual sobre features previas a la activación. Texturas notablemente más realistas.

Real-ESRGAN (2021) — el que usa el notebook

Adapta ESRGAN al **mundo real** combinando:

1. **Degradación de alto orden** (sección 4): ruido + desenfoque + JPEG aplicados en cadena, para que el modelo sepa restaurar fotos reales degradadas, no solo "bicubic".
2. Un **discriminador U-Net con spectral normalization**, que da realimentación por píxel y estabiliza el entrenamiento.

Es el estándar práctico para *upscaling* de fotos y arte (existe **Real-ESRGAN-anime** para ilustración). Pesos x4 listos para usar; soporta *tiling* para imágenes grandes.

👤 ESRGAN: <https://arxiv.org/abs/1809.00219> · Real-ESRGAN: <https://arxiv.org/abs/2107.10833> · Repo: <https://github.com/xinntao/Real-ESRGAN>

8. Transformers para imágenes: ViT, SwinIR y Swin2SR

De ViT al problema de la super-resolución

El **Vision Transformer (ViT, 2020)** trocea la imagen en *parches*, los trata como una secuencia de *tokens* y aplica **auto-atención**: cada token puede "mirar" a todos los demás. La ventaja frente a la convolución es el **contexto global**: la atención modela dependencias de **largo alcance** (relaciona zonas lejanas de la imagen), mientras que una convolución solo ve su vecindario local. Esto conecta directamente con la intuición de "mirar más filas de contexto para decidir mejor un píxel", pero aprendida y sin tabla.

El ViT puro es caro: la atención global escala $O(n^2)$ con el número de tokens, inviable a resolución de imagen completa.

Swin Transformer

Resuelve el coste con **atención por ventanas (windowed attention)**: calcula la atención solo dentro de ventanas locales y las **desplaza** (*shifted windows*) entre capas para que la información fluya entre ventanas. Coste **lineal** con el número de píxeles → aplicable a SR.

SwinIR (2021) y Swin2SR (2022)

- **SwinIR** aplica bloques Swin a la restauración de imagen (SR, *denoising*, *deartifacting*), alcanzando el estado del arte con menos parámetros que las CNN equivalentes.
- **Swin2SR** lo actualiza a **Swin Transformer V2** (mejor estabilidad de entrenamiento y escalado de resolución) y añade un modo *compressed-input* pensado para entradas con JPEG. Está integrado en 🤖

transformers (Swin2SRForImageSuperResolution), por lo que se usa casi igual que un modelo de NLP — justo lo que hace el notebook.

Coste / memoria. La atención, aun por ventanas, consume bastante VRAM con imágenes grandes.

Recomendación: procesar **recortes** o trocear (*tiling*). En el notebook se aplica sobre un recorte pequeño por eso.

🤖 ViT: <https://arxiv.org/abs/2010.11929> · Swin: <https://arxiv.org/abs/2103.14030> · SwinIR: <https://arxiv.org/abs/2108.10257> · Swin2SR: <https://arxiv.org/abs/2209.11345> · Modelos: <https://huggingface.co/caidas>

9. Métricas de calidad y el dilema percepción–distorsión

Comparar métodos de SR no es trivial; conviene reportar varias métricas.

- **PSNR** (Peak Signal-to-Noise Ratio): derivado del MSE. Más alto = más fiel píxel a píxel. No correlaciona bien con la percepción humana (un resultado borroso puede tener alto PSNR).

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

- **SSIM** (Structural Similarity): compara luminancia, contraste y estructura local. Más cercano a la percepción que el PSNR. Rango **[0, 1]**.
- **LPIPS** (Learned Perceptual Image Patch Similarity): distancia en el espacio de features de una red profunda. **Más bajo = más parecido perceptualmente.** Hoy es la métrica perceptual de referencia.
- **NIQE / MANIQA / NRQM**: métricas **sin referencia** (no necesitan la HR original), útiles para SR del mundo real donde no hay *ground truth*.

El dilema percepción–distorsión (Blau & Michaeli, 2018): existe un compromiso **fundamental** entre fidelidad (PSNR/SSIM altos) y realismo perceptual (LPIPS bajo). No se pueden maximizar ambos a la vez. Por eso:

- Los modelos orientados a **PSNR** (EDSR, SwinIR-PSNR) dan resultados fieles pero más suaves.
- Los modelos **GAN/perceptuales** (ESRGAN, Real-ESRGAN) dan texturas nítidas y realistas, pero pueden "inventar" detalle que no estaba (peor PSNR). La elección depende del uso: fidelidad métrica (medicina, forense) vs. estética (fotografía, restauración).

🤖 The Perception-Distortion Tradeoff: <https://arxiv.org/abs/1711.06077> · LPIPS: <https://arxiv.org/abs/1801.03924>

10. Notas prácticas (Colab / GPU T4)

- **¿T4 suficiente?** Sí. Sus 16 GB de VRAM bastan para Real-ESRGAN x4 y Swin2SR sobre imágenes/recortes razonables. Activa la GPU en Colab: *Entorno de ejecución* → *Cambiar tipo de entorno* → *T4 GPU*.
- **Real-ESRGAN** y memoria: para imágenes grandes usa **tiling** (procesa la imagen por baldosas con solape). El repo expone un parámetro **tile** para ello.
- **Swin2SR** es el más sensible a la VRAM (atención $\propto$ píxeles). Mantén la entrada por debajo de $\sim 512 \times 512$ px en una T4, o trocea. En el notebook se usa un recorte de 96 px → x4.

- **Espacio de color.** Mucha SR clásica opera sobre el canal de **luminancia (Y de YCbCr)**, donde vive el detalle que percibe el ojo, y amplía la crominancia por separado. Es la versión "buena" de la intuición de "separar luces/sombras del color" del planteamiento original.
 - **Factor de escala.** Los modelos están entrenados para un **s** concreto (x2, x4...). Usar el modelo correcto importa; encadenar x2 dos veces no equivale a un x4 entrenado.
 - **Formato de entrada.** Da al modelo la imagen **LR real**; no le pases una ya ampliada con bicúbica (salvo modelos que lo esperan), o degradarás el resultado.
-

11. Referencias

Interpolación clásica

- Lanczos resampling — <https://mazzo.li/posts/lanczos.html>
- Keys, *Cubic convolution interpolation* (1981).

SR basada en ejemplos

- Freeman, Jones, Pasztor, *Example-Based Super-Resolution* (2002).
- Yang et al., *Image Super-Resolution via Sparse Representation* (2010).
- Romano, Isidoro, Milanfar, **RAISR** (2016) — <https://arxiv.org/abs/1606.01299>

SR con CNN

- Dong et al., **SRCNN** (2014) — <https://arxiv.org/abs/1501.00092>
- Shi et al., **ESPCN** / sub-pixel conv (2016) — <https://arxiv.org/abs/1609.05158>
- Kim et al., **VDSR** (2016) — <https://arxiv.org/abs/1511.04587>
- Lim et al., **EDSR** (2017) — <https://arxiv.org/abs/1707.02921>
- Zhang et al., **RCAN** (2018) — <https://arxiv.org/abs/1807.02758>

SR perceptual / GAN

- Ledig et al., **SRGAN** (2017) — <https://arxiv.org/abs/1609.04802>
- Wang et al., **ESRGAN** (2018) — <https://arxiv.org/abs/1809.00219>
- Wang et al., **Real-ESRGAN** (2021) — <https://arxiv.org/abs/2107.10833> · <https://github.com/xinntao/Real-ESRGAN>
- Zhang et al., **BSRGAN** (2021) — <https://arxiv.org/abs/2103.14006>

Transformers

- Dosovitskiy et al., **ViT** (2020) — <https://arxiv.org/abs/2010.11929>
- Liu et al., **Swin Transformer** (2021) — <https://arxiv.org/abs/2103.14030>
- Liang et al., **SwinIR** (2021) — <https://arxiv.org/abs/2108.10257>
- Conde et al., **Swin2SR** (2022) — <https://arxiv.org/abs/2209.11345> · <https://github.com/mv-lab/swin2sr>

Métricas

- Blau, Michaeli, *The Perception-Distortion Tradeoff* (2018) — <https://arxiv.org/abs/1711.06077>
- Zhang et al., **LPIPS** (2018) — <https://arxiv.org/abs/1801.03924>

Herramientas / modelos preentrenados

- 🧠 Transformers (Swin2SR) — https://huggingface.co/docs/transformers/model_doc/swin2sr
- Modelos `caidas/swin2SR-*` — <https://huggingface.co/caidas>
- Real-ESRGAN (wrapper sencillo) — <https://github.com/ai-forever/Real-ESRGAN>